Roteiro completo de testes do backend

Apresentacao Inteligente Barueri - roteiro consolidado e ordenado para evitar conflitos

Objetivo. Reunir os tres documentos de teste em um unico fluxo, usando as rotas, nomes de campos e regras do backend atual. Os testes destrutivos ficam no final para evitar quebrar os testes anteriores.

1. Pre-condicoes

• Deve existir pelo menos um usuario ativo com head_manager = true.

• O JWT do head-manager sera usado nas operacoes administrativas.

• Use emails e CPFs de teste que ainda nao existam, pois users.email e users.cpf sao unicos no schema atual.

• Guarde os IDs retornados pelas criacoes para usar nas etapas seguintes.

• Nao exclua o time ou o usuario principal antes de terminar os testes de projetos.

```
Usuario administrativo de exemplo
Email: headmanager@teste.com
Senha: senha123
head_manager: true
status: true
```

2. Variaveis recomendadas no Insomnia

```
{{head_manager_token}}
{{user_token}}
{{approver_token}}
{{sector_id}}
{{team_id}}
{{user_comum_id}}
{{user_approver_id}}
{{user_role_comum_id}}
{{user_role_approver_id}}
{{project_id}}
{{family_id}}
{{project_v2_id}}
{{project_v2_final_id}}
```

Use as variaveis acima para evitar fixar IDs como 1, 2, 3 no roteiro.

3. Login do Head-manager

POST	http://localhost:3000/login

```
{
  "login": "headmanager@teste.com",
  "password": "senha123"
}
```

Esperado: receber access_token. Salve como {{head_manager_token}}.

4. Criar setor

POST	http://localhost:3000/sectors

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Tecnologia e Inovacao"
```

```
}
```

Esperado: setor criado. Salve o ID como {{sector_id}}.

5. Criar time

POST	http://localhost:3000/teams

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Desenvolvimento de Sistemas",
  "sector": {{sector_id}}
}
```

Esperado: time criado. Salve o ID como {{team_id}}.

6. Criar usuarios

6.1 Usuario comum

POST	http://localhost:3000/register

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Usuario Comum",
  "email": "usuario.comum.teste.2026@gmail.com",
  "password": "comum123",
  "cpf": "98765432100"
}
```

Esperado: usuario criado com head_manager = false e status = true. Salve {{user_comum_id}}.

6.2 Usuario aprovador

POST	http://localhost:3000/register

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Usuario Aprovador",
  "email": "usuario.aprovador.teste.2026@gmail.com",
  "password": "aprovador123",
  "cpf": "98765432101"
}
```

Esperado: usuario criado com head_manager = false e status = true. Salve {{user_approver_id}}.

7. Login dos usuarios

7.1 Usuario comum

POST	http://localhost:3000/login

```
{
  "login": "usuario.comum.teste.2026@gmail.com",
  "password": "comum123"
}
```

Esperado: access_token. Salve como {{user_token}}.

7.2 Usuario aprovador

POST	http://localhost:3000/login

```
{
  "login": "usuario.aprovador.teste.2026@gmail.com",
  "password": "aprovador123"
```

```
}
```

Esperado: access_token. Salve como {{approver_token}}.

8. Users-role - criar vinculos

Os antigos /users-sectors e /users-teams nao fazem mais parte do backend atual. Use somente /users-role.

8.1 Vincular usuario comum ao setor e time

POST	http://localhost:3000/users-role

```
Authorization: Bearer {{head_manager_token}}

{
  "user": {{user_comum_id}},
  "sector": {{sector_id}},
  "team": {{team_id}},
  "sector_manager": false,
  "team_manager": false,
  "team_approver": false
}
```

Esperado: nova linha em users_role. Salve {{user_role_comum_id}}.

8.2 Vincular usuario aprovador ao setor e time

POST	http://localhost:3000/users-role

```
Authorization: Bearer {{head_manager_token}}

{
  "user": {{user_approver_id}},
  "sector": {{sector_id}},
  "team": {{team_id}},
  "sector_manager": false,
  "team_manager": false,
  "team_approver": true
}
```

Esperado: nova linha em users_role. Salve {{user_role_approver_id}}.

9. Users-role - consultar

GET	http://localhost:3000/users-role

```
Authorization: Bearer {{head_manager_token}}
```

Teste tambem filtros como user, sector, team e team_approver=true.

10. Users-role - promover e retirar funcoes

O POST cria a relacao; o PATCH altera somente as funcoes daquela relacao.

10.1 Promover para team-manager

PATCH	http://localhost:3000/users-role/{{user_role_comum_id}}

```
Authorization: Bearer {{head_manager_token}}

{
  "team_manager": true
}
```

Esperado: mesmo ID da relacao, agora com team_manager = true.

10.2 Retirar team-manager

PATCH	http://localhost:3000/users-role/{{user_role_comum_id}}

```
Authorization: Bearer {{head_manager_token}}

{
  "team_manager": false
}
```

Esperado: team_manager volta para false.

11. Management - consultar usuarios

GET	http://localhost:3000/management

```
Authorization: Bearer {{head_manager_token}}
```

Teste tambem ?id={{user_comum_id}}, ?name=Usuario e ?email=usuario.comum.teste.2026@gmail.com.

12. Management - alterar usuario

PATCH	http://localhost:3000/management/{{user_comum_id}}

```
Authorization: Bearer {{user_token}}

{
  "current_password": "comum123",
  "name": "Usuario Comum - Alterado",
  "email": "usuario.comum.teste.2026@gmail.com",
  "photo": null
}
```

Esperado: nome alterado. CPF permanece inalterado e senha permanece inalterada.

13. Projects - criar primeiro projeto

POST	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}

{
  "name": "Projeto de Tecnologia",
  "user": {{user_comum_id}},
  "team": {{team_id}},
  "family": 0,
  "short_description": "Descricao curta do projeto",
  "full_description": "Descricao completa do projeto",
  "source": ["https://barueri.sp.gov.br", "https://www.gov.br"],
  "main_department": "Secretaria de Educacao",
  "related_departments": ["Secretaria de Educacao"],
  "areas": ["Tecnologia"],
  "ods": [1, 2],
  "theme": "Educacao e Tecnologia",
  "audience": "Alunos e Professores",
  "technologies": ["React", "NestJS", "PostgreSQL"],
  "implementation_date": "2026-09-04",
  "awards": [
    {
      "institution": "Premio de Inovacao",
      "year": 2026,
      "description": "Premio de inovacao tecnologica",
      "link": "https://exemplo.com"
    }
  ],
  "indicators": [
    {
      "label": "Usuarios atendidos",
      "value": "1000",
      "measure": "pessoas",
      "source": "Secretaria de Educacao",
      "year": 2026
```

```
    }
  ],
  "keywords": ["educacao", "tecnologia"]
}
```

Esperado: status = 1, version = 1 e family = ID do proprio projeto. Salve {{project_id}} e {{family_id}}. Deve gerar log action = "criou".

14. Projects - editar rascunho

PATCH	http://localhost:3000/projects/{{project_id}}

```
Authorization: Bearer {{user_token}}

{
  "name": "Projeto de Tecnologia - Editado",
  "user": {{user_comum_id}},
  "team": {{team_id}},
  "family": 0,
  "short_description": "Descricao curta editada",
  "full_description": "Descricao completa editada",
  "source": ["https://barueri.sp.gov.br"],
  "main_department": "Secretaria de Educacao",
  "related_departments": ["Secretaria de Educacao"],
  "areas": ["Tecnologia"],
  "ods": [1, 2],
  "theme": "Educacao e Tecnologia",
  "audience": "Alunos e Professores",
  "technologies": ["React", "NestJS"],
  "implementation_date": "2026-09-04",
  "awards": [],
  "indicators": [
    {
      "label": "Usuarios atendidos",
      "value": "1200",
      "measure": "pessoas",
      "source": "Secretaria de Educacao",
      "year": 2026
    }
  ],
  "keywords": ["educacao", "tecnologia"]
}
```

Esperado: mesmo id, team, family, version e status. last_update muda e deve ser criado log action = "editou".

15. Projects - impedir nova versao enquanto a ultima esta em rascunho

POST	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}

Use o mesmo family da primeira versao:

"family": {{family_id}}
```

Esperado: a operacao deve falhar porque a ultima versao da familia esta com status = 1. Mensagem esperada: "Projetos em rascunho nao podem ser atualizados".

16. Projects - aprovar

PATCH	http://localhost:3000/projects/{{project_id}}/approve

```
Authorization: Bearer {{approver_token}}
```

Esperado: status 1 -> 2, last_update atualizado e log action = "aprovou".

17. Projects - criar nova versao

POST	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}

{
  "name": "Projeto de Tecnologia - Versao 2",
  "user": {{user_comum_id}},
  "team": {{team_id}},
  "family": {{family_id}},
  "short_description": "Descricao curta da versao 2",
  "full_description": "Descricao completa da versao 2",
  "source": ["https://barueri.sp.gov.br", "https://www.gov.br"],
  "main_department": "Secretaria de Educacao",
  "related_departments": ["Secretaria de Educacao"],
  "areas": ["Tecnologia"],
  "ods": [1, 2, 3],
  "theme": "Educacao e Tecnologia",
  "audience": "Alunos e Professores",
  "technologies": ["React", "NestJS", "PostgreSQL", "TypeScript"],
  "implementation_date": "2026-09-04",
  "awards": [],
  "indicators": [],
  "keywords": ["educacao", "tecnologia", "inovacao"]
}
```

Esperado: nova versao com version = 2, mesmo family e status = 1. Salve {{project_v2_id}}. Deve gerar log action = "criou".

18. Projects - rejeitar nova versao

DELETE	http://localhost:3000/projects/{{project_v2_id}}/reject

```
Authorization: Bearer {{approver_token}}
```

Esperado: projeto removido. awards, indicators e projects_logs vinculados sao removidos por Cascade. A rejeicao nao gera um novo log.

19. Projects - criar novamente a versao 2

POST	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}

Use novamente:
"family": {{family_id}}

Os demais campos podem ser os mesmos da etapa 17.
```

Salve {{project_v2_final_id}}.

20. Projects - aprovar e inativar versao 2

20.1 Aprovar

PATCH	http://localhost:3000/projects/{{project_v2_final_id}}/approve

```
Authorization: Bearer {{approver_token}}
```

Esperado: status 1 -> 2 e log action = "aprovou".

20.2 Inativar

PATCH	http://localhost:3000/projects/{{project_v2_final_id}}/deactivate

```
Authorization: Bearer {{approver_token}}
```

Esperado: status 2 -> 3, last_update atualizado e log action = "inativou".

21. Projects - consultas e filtros

GET	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}
```

Teste individualmente: id, name, user, team, family, main_department, related_departments, areas, ods, theme, status, implementation-date, awards e keywords.

Para arrays, confirme o comportamento AND nos valores. Para awards e keywords, confirme o tratamento de espacos nas extremidades e diferenca de maiusculas/minusculas. Confira tambem source com URL e nome dinamico da fonte.

22. Projects-logs - consultar

GET	http://localhost:3000/projects-logs

```
Authorization: Bearer {{head_manager_token}}
```

Confira os logs de "criou", "editou", "aprovou" e "inativou". A rejeicao nao deve criar um novo log e os logs ligados ao projeto rejeitado sao removidos por Cascade.

23. Users-role - remover relacao

DELETE	http://localhost:3000/users-role/{{user_role_comum_id}}

```
Authorization: Bearer {{head_manager_token}}
```

Esperado: somente a linha de users_role e removida; o usuario permanece em users e outras relacoes podem continuar existindo.

24. Management - soft delete

DELETE	http://localhost:3000/management/{{user_comum_id}}

```
Authorization: Bearer {{head_manager_token}}
```

```
{
  "password": "senha123"
}
```

Esperado: a senha informada e a do head-manager autenticado. O usuario alvo nao e apagado; status passa para false e o id permanece.

25. Confirmar bloqueio do usuario desativado

POST	http://localhost:3000/login

```
{
  "login": "usuario.comum.teste.2026@gmail.com",
  "password": "comum123"
}
```

Esperado: 401 Unauthorized com usuario desativado.

GET	http://localhost:3000/projects

```
Authorization: Bearer {{user_token}}
```

Esperado: o JWT antigo tambem nao deve funcionar, pois o JwtStrategy verifica users.status.

26. Testes destrutivos de setor e time

Execute somente no final e usando dados criados exclusivamente para este teste. Excluir um time pode apagar projetos por Cascade; excluir um setor pode excluir times e, por Cascade, projetos.

26.1 Criar setor de teste

POST	http://localhost:3000/sectors

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Setor Exclusao Teste"
}
```

Salve {{sector_delete_id}}.

26.2 Criar time de teste

POST	http://localhost:3000/teams

```
Authorization: Bearer {{head_manager_token}}

{
  "name": "Time Exclusao Teste",
  "sector": {{sector_delete_id}}
}
```

Salve {{team_delete_id}}.

26.3 Excluir time de teste

DELETE	http://localhost:3000/teams/{{team_delete_id}}

```
Authorization: Bearer {{head_manager_token}}
```

Esperado: time removido fisicamente.

26.4 Excluir setor de teste

DELETE	http://localhost:3000/sectors/{{sector_delete_id}}

```
Authorization: Bearer {{head_manager_token}}
```

Esperado: setor removido fisicamente, com os efeitos de Cascade definidos no schema.

27. Testes de permissao

Repita algumas rotas com tokens de usuarios de niveis diferentes para confirmar os 403.

Operacao	Permitido	Resultado esperado
Criar setor	Head-manager	200/201
Criar setor	Usuario comum	403
Criar time	Head-manager	200/201
Criar time no proprio setor	Sector-manager	200/201
Criar time fora do proprio setor	Sector-manager	403
Atribuir sector-manager	Head-manager	permitido
Atribuir team-manager	Head-manager ou sector-manager do setor	permitido
Atribuir team-approver	Head-manager, sector-manager do setor ou team-manager do time	permitido
Remover users_role	Head-manager ou gestor do escopo	permitido

Operacao	Permitido	Resultado esperado
Criar usuario	Head-manager	permitido
Criar usuario	Sector-manager	403
Desativar usuario	Head-manager	permitido
Desativar usuario	Sector-manager	403

28. Ordem resumida de execucao

```
1. Login do head-manager
2. Criar setor
3. Criar time
4. Criar usuarios
5. Login dos usuarios
6. Criar users-role
7. Consultar users-role
8. Promover/retirar funcoes via PATCH users-role
9. Consultar management
10. Editar usuario
11. Criar projeto v1
12. Editar projeto v1
13. Tentar v2 em rascunho -> deve falhar
14. Aprovar v1
15. Criar v2
16. Rejeitar v2
17. Criar v2 novamente
18. Aprovar v2
19. Inativar v2
20. Consultar projetos
21. Consultar logs
22. Remover relacao users-role
23. Soft delete do usuario
24. Testar login/JWT do usuario desativado
25. Testes destrutivos de setor/time
26. Testes finais de permissao
```

29. Observacoes importantes

• Nao use /users-sectors nem /users-teams: o backend atual usa /users-role.

• Nao use master_admin, institute_manager, area_manager, approver ou level_acess: esses nomes pertencem ao modelo antigo.

• No projeto atual, o novo projeto usa family = 0 para a primeira versao; depois family recebe o ID do proprio projeto.

• A rejeicao de projeto usa DELETE, remove o projeto fisicamente e depende do Cascade para awards, indicators e projects_logs.

• O soft delete de usuario usa status = false e nao remove fisicamente a linha de users.

• CPF nao faz parte do PATCH de management e deve permanecer imutavel.

• O campo head_manager continua sem regra de atribuicao definida, conforme a decisao atual do projeto.